EpiScope: A Big Data Platform for Disease Surveillance, Forecasting and Response Planning in Bangladesh

Group [TEAMMATE: number]: Shezan [ID], Rafi [ID], Rimjhim [ID], Samira [ID]
CSE 4345 Big Data Analytics, Premier University, Chittagong
Repository: https://github.com/Shinzuu/episcope Video: [link added at v1.0]

***Abstract*—EpiScope turns routine health, climate and logistics data into district-level early warnings, one to four week forecasts, an allocation of a fixed national stock and a seasonal preparedness calendar. It is built as a Spark and Delta Lake medallion lakehouse over a synthetic encounter feed of more than 25 million records that takes its population, geography and weather from real public data. This draft fills the data engineering sections from the code and the evidence that exists on 20 October 2026; results marked [P] come from a small SF=0.05 test run and will be replaced by the tagged v1.0 run.**

TEAMMATE: Samira (report lead)

Draft v0.1. Each yellow box names who writes or confirms that part. Every number must come from a file in `docs/experiments/` of the v1.0 run. Page limit: 15 pages plus appendices.

I. Introduction and Problem Analysis

Bangladesh carries a heavy and strongly seasonal load of infectious disease. The 2023 dengue season was the worst on record: WHO counted 308,167 cases and 1,598 deaths up to the end of November [1]. Data that could warn earlier already exists, but it sits in separate systems that differ in format, timeliness and quality, so most analysis happens after hospitals are already full.

The Ministry of Health (a teaching case) asks for a prototype that, for each of the 64 districts, (i) detects unusual rises in dengue, diarrhoea, influenza-like illness (ILI) and malaria, (ii) forecasts cases and hospital demand one to four weeks ahead, (iii) allocates a limited national stock, and (iv) gives a preparedness calendar for the next year. The users are not data scientists, the data is late and incomplete, no patient may be exposed, and the system must run on a 16 GB laptop.

TEAMMATE: Samira

Stakeholder table (MoH, epidemiologists, district officers, hospital managers, vector control, public, regulator) with each one's need and concern, and the risk register (at least 5 risks; start from `docs/proposal.md` §6 and update likelihood and status).

II. CEP Attributes and Conflict Resolution

TEAMMATE: Samira, with each lead for their conflict

P1 to P7, each tied to a design decision. Suggested mapping from the code: P1 depth (Spark, Delta, HMAC, EARS, NB-GLM, LP); P3 (threshold sweep, backtest); P4 unfamiliar issues (late data via MERGE, the silent facility, duplicates across facilities, Dhaka skew); P5 (ICD-10 buckets, IDSR); P6 stakeholders; P7 the dependency chain bronze to dashboard.

C4 freshness against cost. We load once a day in batch. Forecasts and allocations are weekly decisions, and a daily batch over the new files of that day takes minutes, while an always-on stream would hold memory that the 16 GB laptop needs for Spark. Re-running a day costs nothing because bronze skips files it already holds (Section IV).

TEAMMATE: Shezan

Update C4 once the ING-6 streaming path is measured (alert latency against the 5-minute target).

TEAMMATE: Rafi (C1, C3, C5), Rimjhim (C2)

C1: trade-off curve and operating point. C2: suppression policy (counts 1 to 4 hidden, complementary suppression across diseases and weeks). C3: efficiency and equity curve and the recommended α. C5: why the NB-GLM over a more complex model.

III. Architecture and Design Decisions

EpiScope follows the brief's reference architecture: a medallion lakehouse (bronze, silver, gold) on Apache Spark 4.0.1 [2] in local mode with Delta Lake 4.0.1 [3], analytics jobs that read gold and write gold, a serving step that exports suppressed JSON, and a D3.js v7 [4] dashboard. One command, `make all`, runs every stage in order (`reference`, `generate`, `bronze`, `silver`, `gold`, `analytics`, `serve`, `catalog`); Java 17 and Python 3.12 come from `mise.toml` and every Python package is pinned in `requirements.txt`.

Delta rather than plain Parquet. Late records (up to 21 days) must update their epi-week without duplicates. Delta's ACID MERGE makes that a single atomic statement, `replaceWhere` rewrites only the changed weeks of gold, and the transaction log gives time travel for "what did we know on date t". Iceberg would also work; Delta has the documented pairing with Spark 4.0.1 and pip packaging.

Local filesystem rather than HDFS or an object store (ING-5 not done). A Docker HDFS or S3 store costs 1–2 GB of RAM that Spark needs on the target laptop. All paths come from two environment variables, so the lake can move to another URI without code changes.

Partitioning. Bronze is partitioned by `batch_id`, because silver reads exactly one new batch at a time. Silver encounters and gold `cases_weekly` are partitioned by `epi_year` only: one year of gold holds at most $64 \times 4 \times 53$ rows, so a finer key (district or week) would create thousands of tiny files at SF=1 without saving any reads. The proposal planned `division` as a second key; we dropped it for the same reason.

Catalog. The jobs read and write by path, and a separate `make catalog` step registers every table as an external table (`bronze.encounters`, `gold.cases_weekly`, ...) in a Hive metastore stored inside the lake. Derby allows one process at a time, so keeping the catalog out of the jobs means a locked metastore can never stop the pipeline.

TEAMMATE: Samira

Architecture figure (redraw Fig. 1 of the brief with our components) and the alternatives table (Airflow against a Makefile DAG, Iceberg against Delta, streaming against batch).

IV. Data Generation, Quality, Lineage and Privacy

A. Public sources and the generator

District boundaries, codes and the 2022 census population by age band and sex come from the HDX COD-AB and COD-PS datasets [5]; daily precipitation, temperature and humidity at each district centroid for 2021–2025 come from the Open-Meteo archive [6]. Open-Meteo allows about 5,000 weighted calls an hour and 64 districts over five years cost about 8,400, so the download is throttled and the processed reference files (0.6 MB) are committed. A clean clone therefore skips a two-hour download.

The generator (`generator/`) writes the three synthetic sources: encounters (S4) as one CSV folder per report date, a register of 736 facilities (S5) and daily stock snapshots (S6) as JSON. It is vectorised numpy, seeded per (seed, stream, district, year), so the same arguments give byte-identical output whatever the number of worker processes. Rates are per head of population, so the row count follows the census rather than a constant: SF=1 gives about 6.2 million encounters a year, or about 31 million for 2021–2025.

Each disease has its own annual cycle. Dengue peaks in early October, because OpenDengue's national counts peak in October in 2022 and 2024, later than "follows the monsoon" suggests. Dengue responds to rainfall and temperature four to eight weeks earlier, diarrhoea spikes after extreme rainfall, malaria is concentrated in the three Hill Tracts districts and Cox's Bazar, and every count has negative-binomial noise. Reporting delay is a discretised negative binomial with mean 3 days truncated at 21, and 65% of Friday and Saturday reports move to Sunday (Bangladesh's weekend). The SF=0.05 run injected 35 outbreaks[P] of different shape and size, including a slow-growing one and a nationwide novel respiratory surge, and two reporting artefacts: a facility that goes silent for three weeks and then back-fills. They are written only to `ground_truth/`, which the detection code never reads (a test fails if it does).

The generator also plants the defects the quality rules must catch: 2% repeated `encounter_id`, 0.5% orphan `facility_id`, 0.5% invalid age, 0.5% invalid sex, 0.3% malformed ICD-10 codes and 0.3% onsets after the report date. A 14-day incremental drop (`make drop`) uses the same layout and carries records with onsets up to 21 days before the drop.

B. Bronze: keep what arrived

`pipeline/bronze/ingest.py` finds files by glob, never by name, date or count. It computes each file's SHA-256 and skips any file whose checksum is already in the ingest log, so a re-run reads nothing and a new drop loads only its own files. Every value of the contract fields is kept as a string, exactly as written, and each row gets the source, its file, the checksum, the batch ID and the load time (ING-1). One run is one batch; batch IDs start with a UTC timestamp, so they sort in load order.

C. Silver: rules, quarantine and de-identification

The rule catalogue (`pipeline/quality/rules.py`) has 14 rules for encounters and 7 each for facilities and stock (DQ-1). Each rule is a Spark column expression that is true when a row breaks it, with a stable reason code, a severity (quarantine or warn) and the requirement it serves. The six planted defects map one to one onto six rules:

- `DQ_DUP_ENCOUNTER` and `DQ_ORPHAN_FACILITY`;
- `DQ_INVALID_AGE` and `DQ_INVALID_SEX`;
- `DQ_ONSET_AFTER_REPORT` and `DQ_BAD_ICD10`.

The rest cover nulls, types, long delays, future dates and outcome values. Spark 4 runs in ANSI mode, where a plain cast of a bad string throws, so every rule reads values through `try_cast` and an unparseable value is a defect of its own. A failing row goes to `silver.quarantine` once, with the list of every code it broke. In the first SF=0.05 load, 66,979 of 1,681,896 encounter rows (4.0%)[P] were quarantined. Each load writes one row of rule pass rates to `gold.dq_report`, which the dashboard shows (DQ-2).

Before a clean row is stored, `patient_id` becomes an HMAC-SHA256 under a key held only in `.env` (PRV-1). Spark has no HMAC function and a Python UDF over 25 million rows is slow, so the HMAC is built from Spark's native `sha2` with the RFC 2104 inner and outer pads. A plain salted hash would not be enough: national IDs are few enough to enumerate, so the secret key is the only protection. The code refuses to run with a missing key or the placeholder from `.env.example`. Exact age becomes an age band.

D. Idempotent incremental loads (ING-4)

Clean rows are merged into `silver.encounters` with a Delta MERGE on `encounter_id`. A matched row is replaced only when the incoming `report_date` is later. That one condition gives both required properties: re-running a batch matches every row with an equal date and changes nothing, and a resubmitted or late record replaces the old copy and moves to its own onset week. The MERGE is limited to the epi-years in the batch (plus one either side), so partition pruning keeps it from scanning the whole table.

Silver records the weeks each batch changed: the onset weeks of new and replaced rows and the old weeks of replaced rows. Gold rebuilds only those weeks, plus the week after each, with a Delta `replaceWhere`.

E. Gold, lineage and patient de-duplication

`gold.cases_weekly` counts cases, admissions, deaths and lab confirmations per district, disease and ISO week of *onset*. Counting by onset rather than report date is what keeps the back-filled weeks of the silent facility from looking like an outbreak. Every district-week with at least one reporting facility gets a row for all four diseases, so a missing row means "nothing reported" rather than zero, which the detector uses to treat silent weeks as missing (AN-3). Each row carries the sorted bronze batch IDs of the records it counts, which link through the ingest log to file names and checksums (DQ-3).

Cross-facility patient de-duplication (DQ-4) flags a record as a repeat when the same patient key, disease, sex and compatible age band appear at another facility within seven days. On SF=0.05 it flagged 441 pairs, of which 440 were true repeats (precision 0.998)[P]. Recall within the window was only 0.11[P], because the generator draws age and sex afresh for a repeat visit, so most planted repeats fail the attribute check. We kept the strict rule: a false merge would hide a real case.

F. Hidden-drop drill

`scripts/hidden_drop_drill.sh` rehearses the instructor's test: clone, `make setup`, `make all`, a new 14-day drop with an unseen seed, the incremental steps, then the same steps again. On 11 October at SF=0.05 it passed every check[P]: the drop was one new bronze batch, silver gained 8,292 rows, the data moved from 31 December 2025 to 14 January 2026, the re-run changed nothing and left zero duplicate IDs and zero duplicate gold keys, and the export was 9.64 MB with no count from 1 to 4. It took 22 minutes, including setup.

V. Analytics Methods

The code below exists and is tested; the leads write the method justification.

Detection (AN-1, AN-3). EARS C2 [7] runs on seasonally adjusted, completeness-scaled weekly counts with Spark window functions; CUSUM runs per district and disease in `applyInPandas`. Farrington [8] was ruled out because it needs about five years of history before its first test week. Silent facility-weeks are treated as missing, not zero.

Forecasting (AN-4). One negative-binomial GLM per district for dengue and diarrhoea, with weather lagged at least as far as the horizon, fitted in parallel through a cogrouped `applyInPandas`.

Demand, allocation and calendar (AN-6, AN-7, AN-9). Forecast quantiles become beds, IV saline, NS1 kits, ORS and antimalarials with the assumptions in `analytics/forecast/demand_assumptions.py`; the brief's LP is solved with PuLP 3.3.2 and CBC; STL per division and disease gives the onset and peak weeks.

TEAMMATE: Rafi

Write the justification of each method from the literature, the LP formulation with our normalisation per resource, and what α means for policy. Check the allocation first: in the SF=0.05 run the national net need was 0 for beds, IV saline, NS1 kits and ORS, so the LP had almost nothing to allocate.

VI. Evaluation

A. Detection

TEAMMATE: Samira (AN-2), Rafi (AN-3)

Sensitivity, false alerts per district-year and median days to detection for EARS C2 and CUSUM against the naive detector; the C1 curve with the operating point; the artefact facility with and without our handling. Source: `detection_tradeoff.json` from `make evaluate` on the v1.0 data.

B. Forecast backtest

TEAMMATE: Samira (AN-5)

MAE, 80% coverage and WIS [9] against seasonal naive over 52 rolling origins, per horizon. Source: `backtest_metrics.csv`.

C. Allocation against the population baseline

TEAMMATE: Rafi

Share of need met and the worst-covered district for the LP and the population-proportional baseline at five budgets; the efficiency and equity curve. Source: `allocation_summary.json` of the v1.0 run.

D. Scalability and Spark optimisations

TEAMMATE: Samira (ENG-2, ENG-3)

Runtime per stage and output sizes at SF 0.25, 0.5 and 1 from `scaling.json`, with the hardware; the bottleneck stage from the Spark UI; two optimisations with before and after numbers and screenshots.

VII. Dashboard Design and Usability

The dashboard (`dashboard/`) uses only D3.js v7 and the TopoJSON client, both vendored with subresource-integrity hashes. It loads only the aggregated JSON exports, which must stay under 10 MB: the export keeps the most recent weeks that fit. Five views are linked: a district choropleth of incidence per 100,000 with alert markers, a weekly time series with a brush, forecast and bands, a seasonal heatmap,

an allocation view with a what-if panel, and a ranked early-warning table. Suppressed cells are hatched and every view shows the data-freshness time.

TEAMMATE: Rimjhim

Design rationale (encodings, colour-blind-safe palette, why these aggregation levels), screenshots, and the usability test with at least two non-CS users: tasks, findings, changes made.

VIII. Ethical, Societal and Sustainability Considerations

No real patient data is used; every encounter is synthetic and labelled so. Direct identifiers stop at bronze: silver and gold hold only the keyed hash and age bands, the key never enters Git, and every export hides counts from 1 to 4 with complementary suppression so that a total cannot reveal a hidden cell [10]. One residual risk is known and documented in the code: the padded HMAC key appears in Spark's query plan, so Spark event logs of the silver job must be treated as secret and kept outside the repository.

TEAMMATE: Samira

Equity (what α protects and costs), misuse (stigma of a named district, alert fatigue), the differential-privacy discussion for public releases (PRV-4), the limits of big-data signals [11], and the energy and hardware footprint.

IX. Limitations and Future Work

The data is synthetic, so detection and forecast scores measure how well the methods recover patterns we planted, not real-world accuracy. The storage layer runs on the local filesystem, not HDFS or an object store (ING-5). Patient de-duplication is precise but misses most repeats whose age or sex was recorded differently.

TEAMMATE: All

Add each area's main limitation and one concrete next step.

References

[1] World Health Organization, "Dengue: Bangladesh, disease outbreak news DON498," https://www.who.int/emergencies/disease-outbreak-news/item/2023-DON498, 2023.

[2] M. Zaharia *et al.*, "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, 2016.

[3] M. Armbrust *et al.*, "Delta Lake: High-performance ACID table storage over cloud object stores," *Proceedings of the VLDB Endowment*, vol. 13, no. 12, 2020.

[4] M. Bostock, V. Ogievetsky, and J. Heer, "D3: Data-driven documents," *IEEE Transactions on Visualization and Computer Graphics*, vol. 17, no. 12, 2011.

[5] OCHA ROAP and Bangladesh Bureau of Statistics, "Bangladesh subnational administrative boundaries (COD-AB) and population statistics (COD-PS)," Humanitarian Data Exchange, https://data.humdata.org/dataset/cod-ab-bgd, 2025.

[6] P. Zippenfenig, "Open-Meteo.com weather API," Zenodo, https://open-meteo.com, 2023.

[7] L. Hutwagner, W. Thompson, G. M. Seeman, and T. Treadwell, "The bioterrorism preparedness and response Early Aberration Reporting System (EARS)," *Journal of Urban Health*, vol. 80, no. 1, 2003.

[8] C. P. Farrington, N. J. Andrews, A. D. Beale, and M. A. Catchpole, "A statistical algorithm for the early detection of outbreaks of infectious disease," *Journal of the Royal Statistical Society A*, vol. 159, no. 3, 1996.

[9] J. Bracher, E. L. Ray, T. Gneiting, and N. G. Reich, "Evaluating epidemic forecasts in an interval format," *PLoS Computational Biology*, vol. 17, no. 2, 2021.

[10] L. Sweeney, "k-anonymity: A model for protecting privacy," *International Journal of Uncertainty, Fuzziness and Knowledge-Based Systems*, vol. 10, no. 5, 2002.

[11] D. Lazer, R. Kennedy, G. King, and A. Vespignani, "The parable of Google Flu: traps in big data analysis," *Science*, vol. 343, no. 6176, 2014.

Appendix A
Policy Brief for the Director

TEAMMATE: Samira, two pages, no jargon

Current situation; top three recommendations with expected impact; how to read the dashboard; confidence and limitations; data-protection assurance.

Appendix B
Contribution Matrix

TEAMMATE: All, agreed and signed

Work areas from Appendix B of the brief, one column per member (L, S or none), agreed share σ_i (total 100), commits and lines from `git shortlog -sn`, signatures. Ask the instructor to confirm that the moderation factor uses $N = 4$.

Appendix C
Individual Reflections

TEAMMATE: Each member writes their own half page

Shezan, Rafi, Rimjhim, Samira.